OPERATING SYSTEMS & LINUX REVISION GUIDE

A Mechanism-Driven Handbook on Kernel Foundations, Memory Paging, Concurrency & Linux Internals

A Concise, Visual, and Mechanism-Focused Interview & Revision Handbook

METADATA FIELD	SPECIFICATION
Subject:	Systems Engineering & Linux Internals
Target Audience:	SDE / Technical Interviews (Top Tech, FAANG, Product Firms)
Methodology:	Internal Mechanics • State Transitions • Verified Code Recipes • Trade-offs
Source File Consolidated:	Interview Concepts/OS&Linux.py; (926 lines of notes, commands & interview Qs)
Subject Coverage:	Kernel Architecture, Dual-Mode, CPU Scheduling, Concurrency, Paging, Linux Syscalls & Inodes
Key Architectural Focus:	Hardware-OS-User Boundaries, State Machines, Gantt Calculations, Memory Formulas & Invariants
Target Evaluation:	Technical Interview Placement Preparation - Core Engineering Track

CH	CHAPTER TITLE & CORE MECHANISM	CRITICAL INTERVIEW CONCEPTS & FORMULAS
1	OS Architecture & Dual-Mode Execution	User vs Kernel Mode, System Call Trap, Interrupts vs Exceptions
2	Process Lifecycle & Process Control Block	5-State Model, PCB Context, Context Switch Overhead, Zombie vs Orphan
3	Threads, Concurrency & Parallelism	Process vs Thread, ULT vs KLT, Amdahl's Law, Multicore Scaling
4	CPU Scheduling Algorithms & Gantt Analysis	Preemptive vs Non-Preemptive, FCFS, SJF/SRTF, Round Robin Quantum
5	Synchronization & Critical Section Problem	Race Conditions, Mutex vs Counting Semaphore, Monitors, Spinlocks
6	Deadlocks: Detection, Prevention & Avoidance	4 Coffman Conditions, Banker's Algorithm Safe State, Starvation vs Livelock
7	Memory Management, Paging & TLB Translation	Virtual to Physical Address, Page Table Entries, TLB EAT Calculation
8	Virtual Memory, Page Replacement & Thrashing	Page Fault Sequence, FIFO vs LRU vs Optimal, Belady's Anomaly
9	Linux System Calls, Signals & I/O Multiplexing	fork/exec/wait, SIGTERM vs SIGKILL, select vs poll vs epoll O(1)
10	Linux File System, Inodes, Permissions & Links	File Descriptors (0,1,2), Inode Blocks, chmod Octal, Hard vs Soft Links
11	Operating System Archetypes	Multiprogramming, Multitasking, Hard vs Soft RTOS, Distributed

CHAPTER 1 - OS ARCHITECTURE & DUAL-MODE EXECUTION

The Operating System acts as a resource allocator and extended machine interface between hardware and application software. To protect hardware and prevent malicious or runaway processes from crashing the entire system, modern CPUs enforce hardware-level privilege rings.

1.1 Dual-Mode Operation: User Mode vs Kernel Mode

Dimension	User Mode (Ring 3)	Kernel Mode (Ring 0 / Supervisor)
Privilege Level	Restricted; non-privileged CPU instructions only.	Full, unrestricted access to all CPU instructions & hardware.
Memory Access	Confined to the process's own virtual address space.	Access to all physical memory, page tables, and hardware registers.
Hardware Access	Direct I/O access is strictly blocked by hardware.	Direct execution of device I/O, disk writes, and bus signals.
Crash Impact	Process crashes cleanly (SIGSEGV); OS remains stable.	System crash (Kernel Panic in Linux, Blue Screen of Death in Windows).
Transition Mechanism	Triggers a software trap/interrupt (syscall instruction).	Returns to User Mode via sysret or iret instruction.

1.2 The System Call Lifecycle & Trap Mechanism

A **System Call** is the controlled programmatic gateway through which a user process requests kernel services.
Exact Execution Flow:
1. **User Invocation:** Application calls a library wrapper (e.g., read(fd, buf, count)).
2. **Trap Generation:** Wrapper places the syscall number in register (e.g., %rax on x86_64) and executes the syscall assembly instruction.
3. **Mode Switch:** CPU hardware flips the mode bit from 1 (User) to 0 (Kernel) and jumps to the fixed kernel entry point defined in the Interrupt Descriptor Table (IDT).
4. **Dispatch:** Kernel indexes into the Syscall Dispatch Table, validates buffer pointers, and executes the driver routine.
5. **Return:** Result is stored in register; CPU executes sysret, restoring user context and mode bit to 1.

1.3 Interrupts vs Exceptions vs System Calls

Event Type	Origin / Cause	Synchronous?	Handling Flow
Hardware Interrupt	External device (Keyboard, Timer tick, NIC packet arrival).	Asynchronous (can happen at any clock cycle)	CPU pauses current instruction, saves PC, executes ISR, resumes.
Software Exception	Internal CPU execution error (Divide-by-zero, Page Fault).	Synchronous (tied directly to current instruction)	Kernel handles fault or sends signal (SIGFPE, SIGSEGV) to process.
System Call (Trap)	Voluntary software request for kernel service.	Synchronous (explicitly programmed)	Kernel switches to Ring 0, dispatches routine, returns result.

CHAPTER 2 - PROCESS LIFECYCLE & PROCESS CONTROL BLOCK

A **Program** is a passive entity (an executable file on disk). A **Process** is an active entity: a program in execution with an assigned Program Counter, stack, heap, and register state.

2.1 The 5-State Process Model & Transitions

- **1. NEW:** The process is being created (memory allocated, PCB initialized).
- **2. READY:** The process is in RAM, loaded with all required resources, waiting in the ready queue for CPU allocation.
- **3. RUNNING:** The CPU scheduler assigns a core; CPU registers execute process instructions.
- **4. WAITING (BLOCKED):** The process requested I/O, sleep, or a mutex lock; cannot use CPU even if available.
- **5. TERMINATED:** Execution completed or killed; resources freed, waiting for parent to reap exit code.

STATE TRANSITION RULES

- **Ready -> Running:** Scheduled by CPU Scheduler.
- **Running -> Ready:** Preempted by Timer Interrupt (quantum expired) or higher-priority process.
- **Running -> Waiting:** Initiates blocking I/O or waits for child/event.
- **Waiting -> Ready:** I/O completes or signal arrives (NEVER transitions directly from Waiting to Running!).

2.2 Anatomy of the Process Control Block (PCB)

The PCB (represented by task_struct in the Linux kernel) is the data structure maintaining complete process context: PID, PPID, Process State, Program Counter (PC), CPU Registers, CPU Scheduling Info (priority), Memory Management Info (Page table pointers, mm_struct), and Open File Descriptor Table.

2.3 Context Switching Overhead & Cost

When switching from Process A to Process B:
1. Save CPU registers and PC of Process A into PCB_A.
2. Switch memory maps (update CR3 register to point to Process B's page table).
3. **The Hidden Cost:** Invalidates the CPU Translation Lookaside Buffer (TLB) and causes L1/L2/L3 cache misses! Context switching is pure administrative overhead where zero useful computation occurs.

2.4 Zombie vs Orphan Processes

Process State	Definition & Cause	Resource Leaked?	Resolution Method
Zombie Process	Terminated child whose parent has not yet called wait().	Memory is freed, but PCB and PID entry remain in Process Table.	Parent must call wait(); if parent dies, init/systemd reaps it.
Orphan Process	Running child whose parent process died or terminated first.	No leak; child continues normal execution.	Automatically adopted by init / systemd (PID 1).

CHAPTER 3 - THREADS, CONCURRENCY & PARALLELISM

A **Thread** is the smallest schedulable unit of CPU execution within a process. A single process can contain multiple threads that execute concurrently.

3.1 Process vs Thread: Memory & Isolation Architecture

Dimension	Process (Heavyweight)	Thread (Lightweight)
Address Space	Independent, isolated virtual memory space.	Shared text (code), data, and heap with peer threads.
Private Resources	Entire address space, file descriptors, signals.	Only private Stack, Program Counter (PC), and CPU Registers.
Creation Overhead	High (requires duplicating page tables, allocating PCB).	Low (shares memory map; allocates only a thread stack).
Context Switch Cost	High (must flush TLB and change page table base register).	Low (memory map unchanged; only switches stack pointer & registers).
Communication	Inter-Process Communication (IPC: pipes, sockets, shared mem).	Direct shared memory reads/writes (requires synchronization!).
Crash Isolation	Isolated: if one process crashes, others continue safely.	Shared: an unhandled fault (SIGSEGV) in one thread kills the whole process.

3.2 User-Level Threads (ULT) vs Kernel-Level Threads (KLT)

• **User-Level Threads (ULT):** Managed entirely in user space by a threading runtime library. The OS kernel is unaware of ULTs and schedules only the parent process. *Advantage:* Ultra-fast context switching (no syscall). *Fatal Flaw:* If one thread makes a blocking system call, the kernel blocks the entire process! Cannot achieve true multi-core parallelism.
• **Kernel-Level Threads (KLT):** Managed directly by the OS kernel (Linux NPTL / clone). The kernel schedules individual threads across separate CPU cores simultaneously. *Advantage:* True multicore parallelism and non-blocking concurrency.

3.3 Concurrency vs Parallelism

• **Concurrency:** Managing multiple tasks at overlapping periods of time. A single CPU core achieves concurrency via rapid time-slicing (interleaving).
• **Parallelism:** Executing multiple tasks at the exact same physical instant. Requires physical multi-core or multiprocessor hardware.
• **Amdahl's Law Formula:** Maximum Speedup = 1 / ((1 - P) + (P / N)), where P is the parallel portion of a program and N is the number of processor cores. The sequential portion (1 - P) forms the asymptotic upper bound on performance scaling.

CHAPTER 4 - CPU SCHEDULING ALGORITHMS & METRICS

The CPU Scheduler decides which ready process receives the CPU core when the current process yields, blocks, or exhausts its time slice.

4.1 Core Performance Metrics

- **Turnaround Time:** Completion Time - Arrival Time (Total time from submission to exit).
- **Waiting Time:** Turnaround Time - Burst Time (Time spent sitting in Ready Queue).
- **Response Time:** First CPU Execution Time - Arrival Time (Crucial for interactive systems).
- **Throughput:** Number of processes completed per unit of time.

4.2 Preemptive vs Non-Preemptive Scheduling Algorithms

Algorithm	Preemptive?	Selection Criteria	Pros & Cons / Convoy Effect
FCFS (First-Come, First-Served)	No	Earliest Arrival Time	Simple, but suffers from **Convoy Effect** (short processes wait behind long burst).
SJF (Shortest Job First)	No	Smallest CPU Burst Time	Provably optimal minimum average waiting time. Starvation of long jobs.
SRTF (Shortest Remaining Time First)	Yes	Smallest Remaining Burst	Preempts if a new arrival has smaller remaining time than running job.
Round Robin (RR)	Yes	FCFS with Fixed Time Quantum (q)	Fair, interactive. If q too large -> degenerates to FCFS. If q too small -> context switch storm!
Priority Scheduling	Both	Highest Assigned Priority	High priority runs first. Suffers from Starvation (solved by **Aging**).

4.3 Worked Example: Round Robin (Quantum = 2)

Given processes: P1 (Burst 5, Arrival 0), P2 (Burst 3, Arrival 1), P3 (Burst 1, Arrival 2).
Gantt Execution Timeline:
[0..2]: P1 (rem 3) | [2..4]: P2 (rem 1) | [4..5]: P3 (DONE!) | [5..7]: P1 (rem 1) | [7..8]: P2 (DONE!) | [8..9]: P1 (DONE!)
- P3 completes at t = 5. Turnaround = 5 - 2 = 3. Waiting = 3 - 1 = 2.
- P2 completes at t = 8. Turnaround = 8 - 1 = 7. Waiting = 7 - 3 = 4.
- P1 completes at t = 9. Turnaround = 9 - 0 = 9. Waiting = 9 - 5 = 4.

Average Waiting Time: (4 + 4 + 2) / 3 = **3.33 units**.

CHAPTER 5 - SYNCHRONIZATION & CRITICAL SECTION PROBLEM

When multiple concurrent threads access and mutate shared data without coordination, execution order depends on non-deterministic thread scheduling. This race condition produces corrupted or inconsistent state.

5.1 The 3 Mandatory Criteria for Critical Section Solutions

1. **Mutual Exclusion:** If process P is executing in its critical section, no other process can execute in its critical section.
2. **Progress:** If no process is in its critical section and some wish to enter, only those not in their remainder section can participate in deciding who enters next, and selection cannot be postponed indefinitely.
3. **Bounded Waiting:** There must be a bound on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter and before that request is granted (prevents starvation).

5.2 Mutex vs Counting Semaphore vs Spinlock

Mechanism	State / Value	Behavior on Contention	Ideal Use Case
Mutex (Lock)	Binary (0 or 1, Locked/Unlocked)	Calling thread sleeps (blocks) until owner releases lock.	Protecting exclusive access to a single shared data structure.
Counting Semaphore	Integer count (N available tokens)	Decrements on wait(); sleeps if count == 0. Increments on signal().	Managing access to a finite pool of identical resources (e.g., DB connections).
Spinlock	Atomic flag (test-and-set)	Busy-waits (loops consuming CPU) until lock becomes free.	Ultra-short critical sections in multicore OS kernel drivers where sleep overhead exceeds wait.

5.3 Semaphores: wait() and signal() Atomic Invariants

Code: Semaphore wait() / signal() Operations

```
# Semaphore Primitives (Must execute atomically via hardware Test-and-Set)
def wait(semaphore):
    semaphore.value -= 1
    if semaphore.value < 0:
        add_calling_thread_to_queue(semaphore.waiting_list)
        block()  # Put thread to sleep

def signal(semaphore):
    semaphore.value += 1
    if semaphore.value <= 0:
        thread = remove_thread_from_queue(semaphore.waiting_list)
        wakeup(thread)  # Move thread from Waiting to Ready queue
```

CHAPTER 6 - DEADLOCKS: CONDITIONS & BANKER'S ALGORITHM

A **Deadlock** is a permanent stall where every process in a set is waiting for an event (resource release) that can only be caused by another process in the set.

6.1 The 4 Coffman Conditions (All 4 Required)

1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode.
2. **Hold and Wait:** A process must be holding at least one resource and waiting to acquire additional resources held by others.
3. **No Preemption:** Resources cannot be forcibly seized; a resource is released only voluntarily by the holding process.
4. **Circular Wait:** A closed chain of processes exists: P0 waits for P1, P1 waits for P2, ..., Pn waits for P0.

DEADLOCK PREVENTION STRATEGY

To guarantee deadlock can never occur, break at least ONE Coffman condition:
• Break Hold & Wait: Require processes to request all resources at once.
• Break No Preemption: Forcibly revoke resources if a new request cannot be granted.
• Break Circular Wait: Impose a strict global numerical ordering on all resource types (F: R -> N). A process can only request resources in strictly ascending order.

6.2 Deadlock Avoidance: Banker's Algorithm

Banker's Algorithm tests whether allocating resources to a request leaves the system in a **Safe State** (a state where at least one safe sequence exists to satisfy maximum remaining needs of all processes).

Code: Banker's Algorithm Implementation

```
# Banker's Algorithm Safe State Verification
# Invariants: Need[i][j] = Max[i][j] - Allocation[i][j]
def is_safe_state(available, max_claim, allocation):
    num_p = len(allocation)
    num_r = len(available)
    need = [[max_claim[i][j] - allocation[i][j] for j in range(num_r)] for i in range(num_p)]

    work = list(available)
    finish = [False] * num_p
    safe_seq = []

    for _ in range(num_p):
        found = False
        for i in range(num_p):
            if not finish[i] and all(need[i][j] <= work[j] for j in range(num_r)):
                # Simulate process completing and releasing all resources
                for j in range(num_r):
                    work[j] += allocation[i][j]
                finish[i] = True
                safe_seq.append(i)
                found = True
                break
        if not found:
            return False, []  # Unsafe state! Potential Deadlock!
    return True, safe_seq
```

CHAPTER 7 - MEMORY MANAGEMENT, PAGING & ADDRESS TRANSLATION

Virtual memory isolates application processes from physical RAM hardware. Programs operate in contiguous virtual address spaces, while physical RAM is divided into discrete fixed-size blocks.

7.1 Paging Mechanics: Virtual Page to Physical Frame

• **Page:** A fixed-size contiguous block of Virtual Memory (typically 4 KB).
• **Frame:** A matching fixed-size contiguous block of Physical RAM.
• **Address Structure:** A Virtual Address consists of [Virtual Page Number (VPN) | Page Offset (d)].
• **Translation Invariant:** The offset d is copied unchanged to the physical address. The Memory Management Unit (MMU) indexes the Page Table using VPN to retrieve the Physical Frame Number (PFN).

7.2 Translation Lookaside Buffer (TLB) & Effective Access Time

Because page tables reside in RAM, every memory reference naively requires two RAM accesses (one to read the page table, one to access data). The TLB is a high-speed associative hardware cache storing recently used VPN -> PFN mappings.

EFFECTIVE ACCESS TIME (EAT) FORMULA

Let **h** = TLB hit ratio, **t_tlb** = TLB lookup time, **t_mem** = Main memory access time.
• **TLB Hit Time:** t_tlb + t_mem (Translation found in cache, single RAM fetch).
• **TLB Miss Time:** t_tlb + t_mem (read page table) + t_mem (fetch data) = t_tlb + 2 * t_mem.
Formula: EAT = h * (t_tlb + t_mem) + (1 - h) * (t_tlb + 2 * t_mem).
Worked Example: If h = 90%, t_tlb = 10ns, t_mem = 100ns:
EAT = 0.90 * (110ns) + 0.10 * (210ns) = 99 + 21 = **120 ns** (vs 200ns un-cached!).

7.3 Internal vs External Fragmentation

Fragmentation Type	Cause	Occurs In?	Solution
Internal Fragmentation	Process payload is smaller than allocated fixed page/block size.	Paging systems (e.g., allocating a 4 KB page for a 1 KB process).	Use smaller page sizes or dynamic slab allocators.
External Fragmentation	Total free memory exceeds request, but is fragmented into non-contiguous slices.	Variable-partition contiguous allocation / Segmentation.	Paging (eliminates external fragmentation entirely!) or memory compaction.

CHAPTER 8 - VIRTUAL MEMORY, PAGE REPLACEMENT & THRASHING

Virtual Memory allows executing processes whose total memory requirements exceed physical RAM capacity by keeping only active pages in RAM.

8.1 The 6-Step Page Fault Handling Workflow

When a CPU references a page whose Valid/Invalid bit in the Page Table is set to 0 (Invalid / Not in RAM):
1. **Trap:** MMU raises a Page Fault Hardware Exception to the OS kernel.
2. **Validation:** Kernel checks internal process tables to confirm address is a valid memory reference (not a segmentation violation).
3. **Victim Selection:** Kernel locates a free physical frame in RAM. If none is free, a Page Replacement Algorithm selects a victim page to evict.
4. **Disk I/O:** If victim page is dirty (modified bit = 1), write it to swap space. Read the requested page from disk into the allocated frame.
5. **Update Table:** Update Page Table entry with new Frame Number and set Valid bit = 1.
6. **Instruction Restart:** Restart the exact CPU instruction that caused the page fault.

8.2 Page Replacement Algorithms

Algorithm	Eviction Rule	Belady's Anomaly?	Practical Implementation
FIFO (First-In, First-Out)	Evicts the page that has resided in RAM the longest.	YES (Belady's Anomaly applies!)	Simple FIFO queue of frame pointers.
LRU (Least Recently Used)	Evicts the page whose last reference is farthest in the past.	NO (Stack algorithm; immune)	Hardware timestamp counter or doubly linked list.
Optimal (OPT / MIN)	Evicts the page that will not be used for the longest future time.	NO (Theoretical benchmark)	Impossible in real time (requires clairvoyant future knowledge).

8.3 Belady's Anomaly & Thrashing

• **Belady's Anomaly:** Counter-intuitive phenomenon where increasing physical frame capacity causes an INCREASE in page faults under FIFO replacement. Immune algorithms (LRU, Optimal) satisfy the *stack inclusion property*: the set of pages in N frames is always a subset of pages in N+1 frames.
• **Thrashing:** Condition where the system spends more time servicing page faults and disk I/O than executing instructions. Occurs when the sum of process **Working Sets** exceeds available physical RAM. *Remedy:* Decrease degree of multiprogramming (suspend low-priority processes) or add physical RAM.

CHAPTER 9 - LINUX SYSTEM CALLS, SIGNALS & I/O MULTIPLEXING

9.1 Process Lifecycle Syscalls: fork(), exec(), wait()

Code: Linux Process Creation (fork + exec + wait)

```
#include <unistd.h>
#include <sys/wait.h>
#include <stdio.h>

int main() {
    pid_t pid = fork();  // Creates exact clone of current process
    if (pid < 0) {
        perror("Fork failed");
    } else if (pid == 0) {
        // Child Process: Returns 0 from fork()
        char *args[] = {"/bin/ls", "-l", NULL};
        execvp(args[0], args);  // Replaces child's address space with new program
    } else {
        // Parent Process: Returns Child PID from fork()
        int status;
        waitpid(pid, &status, 0);  // Blocks until child terminates and reaps PCB
        printf("Child exited successfully.\n");
    }
    return 0;
}
```

9.2 Linux Process Signals & Kill Commands

Signal Name	Signal Number	Catchable / Maskable?	Default Action / Purpose
SIGTERM	15	Yes (Can be caught & handled)	Requests graceful shutdown (flush caches, close sockets, save state).
SIGKILL	9	NO (Cannot be caught or ignored!)	Forcibly terminates process immediately at kernel level.
SIGINT	2	Yes	Interrupt from terminal (Ctrl + C).
SIGSTOP	19	NO	Pauses / suspends process execution (Ctrl + Z sends SIGTSTP).
SIGSEGV	11	Yes	Segmentation fault (invalid virtual memory dereference).

9.3 I/O Multiplexing Evolution: select() vs poll() vs epoll()

Mechanism	Scale Bound	Time Complexity	Kernel Mechanics / Scalability
select()	FD_SETSIZE (1024 max)	O(N) linear scan	Copies entire bitmask back and forth; iterates over all 1024 FDs.
poll()	No hard FD limit	O(N) linear scan	Passes array of pollfd structs; still requires O(N) scan across all FDs.
epoll()	Scales to 100,000+ FDs	**O(1) event-driven**	Uses in-kernel Red-Black Tree and Ready List. Kernel alerts only active FDs via callbacks.

CHAPTER 10 - LINUX FILE SYSTEM, INODES, PERMISSIONS & LINKS

10.1 Inodes & File Metadata

In Linux, a **Directory** is simply a file containing pairs of (filename, inode_number). The file content and metadata are tracked by the **Inode**: file type, permissions (rwx), owner UID/GID, file size, timestamps (atime, mtime, ctime), and data block pointers (direct, single indirect, double indirect, triple indirect pointers). *Crucial Fact:* The filename is NOT stored in the inode; it exists only in directory entries!

10.2 Hard Links vs Soft (Symbolic) Links

Dimension	Hard Link (ln target link)	Soft / Symbolic Link (ln -s target link)
Inode Identity	Points directly to the **same inode number**.	Creates a new inode whose content is the path string to target.
Cross-Filesystem	Cannot cross filesystem boundaries.	Can link across different disks, partitions, and filesystems.
Directory Linking	Prohibited by OS (prevents cyclic directory loops).	Allowed.
Target Deleted	File data remains intact as long as link count > 0.	Becomes a **Dangling / Broken Link** pointing nowhere.

10.3 Octal Permission Calculation & Commands

Code: Linux Permissions & Process Diagnostic Commands

```
# Permissions Representation: User (Owner) | Group | Others
# r = 4, w = 2, x = 1
chmod 755 script.sh   # Owner: rwx (7), Group: r-x (5), Others: r-x (5)
chmod 644 document.txt# Owner: rw- (6), Group: r-- (4), Others: r-- (4)

# Diagnostic Commands:
ps aux | grep python3       # Find PID of running python scripts
kill -9 <PID>               # Send SIGKILL (force terminate)
lsof -i :8080               # List process listening on port 8080
df -h; du -sh /var/log      # Disk filesystem usage and directory consumption
```

CHAPTER 11 - OPERATING SYSTEM ARCHETYPES & ARCHITECTURES

Architecture	Core Design Principle	Scheduling / Execution Model	Typical Application
Multiprogramming	Keep CPU busy at all times by having multiple jobs in RAM.	Switches to another job only when current job blocks on I/O.	Batch processing systems, mainframe workloads.
Multitasking (Time-Sharing)	Provide rapid response time for multiple interactive users.	Preemptive time slicing (CPU allocated for quantum q).	General desktop and server OS (Linux, Windows, macOS).
Hard Real-Time (RTOS)	Strict deterministic timing guarantees; missing a deadline is catastrophic.	Priority-preemptive with zero unbounded kernel latencies.	Pacemakers, anti-lock brakes, avionics, robotics.
Soft Real-Time (RTOS)	Deadlines are prioritized but missing a deadline degrades quality without failure.	Prioritizes multimedia/streaming tasks over background jobs.	Video conferencing, streaming, video gaming engines.

OPERATING SYSTEMS INTERVIEW CHEAT-SHEET

- **Processes vs Threads:** Processes have isolated memory; threads share heap/data but have private stacks.
- **Deadlock 4 Conditions:** Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait.
- **Paging Address Translation:** VPN indexes Page Table to find PFN; page offset d remains identical.
- **Virtual Memory Replacement:** FIFO suffers from Belady's Anomaly; LRU is immune.
- **epoll vs select:** epoll is O(1) event-driven via in-kernel ready list; select is O(N) bitmask scan.